

Recitation 3: Number Theory Review

Instructors: Prof. Goldwasser and Prof. Vaikuntanathan**TAs:** Zoe Xi and Noga Amit**Contents**

1	Groups	2
1.1	Notation	2
1.2	Order of a group and the order of an element	2
1.3	Generators	3
1.4	Cyclic group	3
1.5	Discrete Logarithms	3
2	Baby (Computational) Number Theory	4
2.1	Greatest Common Divisors	4
2.1.1	Euclid's Algorithm	4
2.2	Modular Arithmetic	4
2.3	Modular Exponentiation	4
2.3.1	Chinese Remainder Theorem	5
3	The Multiplicative Group $\mathbb{Z}_N^*$	5
4	The Multiplicative Group $\mathbb{Z}_P^*$ for a Prime P	6
4.1	$\mathbb{Z}_P^*$ has lots of generators	7
4.2	The multiplicative group $\mathbb{Z}_P^*$ and the additive group $\mathbb{Z}_{P-1}$	7
4.3	Primes, Primality Testing and Generating Random Primes	7
4.4	The Discrete Log Problem	8
4.4.1	Solving the Discrete Log Problem	8
4.4.2	Random Self-Reducibility of Discrete Logarithms	9
4.5	Quadratic Residues	10
4.6	Other Roots	11
4.7	The Diffie-Hellman Assumptions	11
4.7.1	DDH is False in the multiplicative group	12

1 Groups

An Abelian group $\mathbb{G} = (S, \star)$ is a set S together with an operation $\star : S \times S \rightarrow S$ which satisfies

- **Identity:** There is an element $\mathcal{I} \in S$ such that for all $a \in S$, $a \star \mathcal{I} = \mathcal{I} \star a = a$.
- **Inverse:** For every $a \in S$, there is an element $b \in S$ such that $a \star b = b \star a = \mathcal{I}$.
- **Associativity:** For every $a, b, c \in S$, $a \star (b \star c) = (a \star b) \star c$.
- **Commutativity:** For every $a, b \in S$, $a \star b = b \star a$.

Notice that we defined Abelian (or commutative) groups by default. Those will be the only type of groups that we see in this class. The definition of general groups is the same as the above, except without the commutativity property. Also, most of the time in this class, the groups we deal with will be finite, namely S will be a finite set. An occasional exception is the additive group $\mathbb{Z}$ of all integers, which is infinite.

1.1 Notation

- We will sometimes abuse notation and say that $g \in \mathbb{G}$ when we really mean that $g \in S$.
- We will denote the multiplicative shorthand for iterated group operations by default. That is, for $g \in S$, $g^2 = g \star g$, $g^3 = g \star g \star g$, and so forth.
- $g^0 = \mathcal{I}$ and g^{-1} denotes the inverse of g . That is, $g(g^{-1}) = \mathcal{I}$

Example 1: The Additive Group $\mathbb{Z}_N$.

A simple example is the additive group $\mathbb{Z}_N := (\{0, 1, 2, \dots, N-1\}, +)$ consisting of integers from 0 to $N-1$, where the group operation is addition modulo N . The identity element is 0, its inverse is 0, and the inverse of $x \neq 0$ is $N-x$. We will call them the additive identity and the additive inverse, to distinguish them from their multiplicative friends who will show up soon, and who will turn out to be more useful.

1.2 Order of a group and the order of an element

The order of a group is the number of elements in it, namely $|S|$. The order of an element $g \in S$ is the number of times one has to perform the group operation on g to get to the identity element $\mathcal{I}$. That is,

$$\text{ord}(g) = \min\{i > 0 : g^i = \mathcal{I}\}.$$

Example 2: Order of $\mathbb{Z}_N$.

The order of the group $\mathbb{Z}_N$ is N , since it contains N elements. Moreover, 1 has order N because

$$\underbrace{1 + 1 + \dots + 1}_{N \text{ times}} = N \pmod{N} = 0 = \mathcal{I} \in \mathbb{Z}_N.$$

Theorem 1 (Lagrange's Theorem). *The order of any element divides the order of the group.*

Proof. Let $g \in \mathbb{G}$ be some group element. Note that multiplication by g defines a bijection on $\mathbb{G}$. That is, $f : \mathbb{G} \rightarrow \mathbb{G}$ defined by $f(x) = gx$ is a one-to-one and onto function.

We first claim that $g^{|\mathbb{G}|} = \mathcal{I}$. Let's multiply all the group elements in two ways. Notice that on the left and the right, we are computing the product over the same set of elements since multiplying by g permutes the elements of the group. So,

$$\prod_{h \in \mathbb{G}} h = \prod_{h \in \mathbb{G}} (hg) = g^{|\mathbb{G}|} \left(\prod_{h \in \mathbb{G}} h \right).$$

Dividing by $\prod_{h \in \mathbb{G}} h$ from the left and the right gives us $g^{|\mathbb{G}|} = \mathcal{I}$.

Now let x be the order of g . Let $x' = \gcd(x, |\mathbb{G}|)$. By the extended Euclidean algorithm (see Section 2), there are integers a and b such that

$$ax + b|\mathbb{G}| = x'.$$

Therefore,

$$g^{x'} = g^{ax+b|\mathbb{G}|} = (g^x)^a (g^{|\mathbb{G}|})^b = (\mathcal{I})^a (\mathcal{I})^b = \mathcal{I}.$$

If $x' < x$, this contradicts the assumption that x was the order of g , and therefore the smallest positive power of g that gives $\mathcal{I}$. So, it must be the case that $x' = x$ and thus $x = \gcd(x, |\mathbb{G}|)$ and consequently, x divides $|\mathbb{G}|$. $\square$

1.3 Generators

A generator of a group $\mathbb{G}$ is an element of order $|\mathbb{G}|$. In other words,

$$\mathbb{G} = \{g, g^2, \dots, g^{|\mathbb{G}|} = \mathcal{I}\}.$$

Example 3: Generator of $\mathbb{Z}_N$.

1 is a generator of $\mathbb{Z}_N$ for any N .

1.4 Cyclic group

A group $\mathbb{G}$ is called cyclic if it has a generator. By the above, $\mathbb{Z}_N$ is always cyclic. We also know:

Theorem 2. *Every group whose order is a prime number, is cyclic. Moreover, every element other than the identity is a generator.*

Proof. Left as an exercise; see also Angluin [2]. $\square$

1.5 Discrete Logarithms

Let $\mathbb{G}$ be a cyclic group and let g be a generator of $\mathbb{G}$. Then every $h \in \mathbb{G}$ can be written as $h = g^x$ for a unique $x \in \{0, 1, \dots, |\mathbb{G}| - 1\}$. We write

$$x = \text{dlog}_g(h)$$

to denote the fact that x is the discrete logarithm of h to the base g .

We will look for groups where computing the group operations is easy (namely, polynomial time) but computing discrete logarithms is hard (namely, exponential or sub-exponential time). Our source for such groups will come from number theory, so let us move on to reviewing a few basic notions from (computational) number theory.

Discrete logarithms in $\mathbb{Z}_N$ are, for better or worse, easy. Indeed, the generators of $\mathbb{Z}_N$ are precisely those g whose greatest common divisor with N is 1. If you are told that

$$x \cdot g = h \pmod{N}$$

then x can be computed as $h \cdot g^{-1} \pmod{N}$ where g^{-1} now denotes the modular multiplicative inverse of g which can be found easily using the extended Euclidean algorithm (see Section 2.) It is at this point that we abandon $\mathbb{Z}_N$ and instead move on to other groups. Specifically, the multiplicative group $\mathbb{Z}_N^*$ for suitable N in which computing the discrete logarithm is conjectured to be computationally intractable.

2 Baby (Computational) Number Theory

2.1 Greatest Common Divisors

The greatest common divisor (gcd) of positive integers a and b is the largest positive integer d that divides both a and b . a and b are called *relatively prime* if their gcd is 1.

2.1.1 Euclid's Algorithm

One can find d using Euclid's algorithm in time $O(n^2)$ where n is the maximum of the bit-lengths of a and b . One can do more: it turns out that there are always integers (not necessarily positive) x and y such that

$$ax + by = d.$$

These integers can be found in essentially the same complexity using a modification of Euclid's basic algorithm, referred to as the extended Euclidean algorithm.

2.2 Modular Arithmetic

For an integer a , $a \pmod{N}$ will denote the remainder upon dividing a by N . We call b the (multiplicative) inverse of a if $ab = 1 \pmod{N}$. In this case, we say $b = a^{-1} \pmod{N}$. The multiplicative inverse of a exists if and only if $\gcd(a, N) = 1$.

Exercise: Show a polynomial-time algorithm to compute the multiplicative inverse of a given number a modulo a given N .

2.3 Modular Exponentiation

Modular exponentiation is just like regular exponentiation except that we compute the final result modulo N . However, computing a^b over the integers and then reducing mod N is a horribly

inefficient way to do this (think about it). Instead, one uses the repeated squaring algorithm (mod N) which reduces the run time down to $O(n^3)$.

2.3.1 Chinese Remainder Theorem

Let $N = \prod_{i=1}^{\ell} P_i^{\alpha_i}$. The Chinese remainder theorem states that the following group isomorphism is true:

$$\mathbb{Z}_N^* \equiv \mathbb{Z}_{P_1^{\alpha_1}}^* \times \cdots \times \mathbb{Z}_{P_{\ell}^{\alpha_{\ell}}}^*.$$

That is, there is an isomorphism ϕ that maps $\mathbb{Z}_N^*$ to a direct product of the groups $\mathbb{Z}_{P_i^{\alpha_i}}^*$. This isomorphism is efficiently computable in both directions. Slightly less abstractly, ϕ maps every element $x \in \mathbb{Z}_N^*$ into a tuple of elements

$$\vec{x} = (x \pmod{P_1^{\alpha_1}}, x \pmod{P_2^{\alpha_2}}, \dots, x \pmod{P_{\ell}^{\alpha_{\ell}}}).$$

Multiplying x and y mod N is equivalent to multiplying $\vec{x}$ and $\vec{y}$ componentwise. ϕ is a one-to-one onto mapping, and ϕ^{-1} can be computed as a linear function: letting $\vec{x} = (x_1, \dots, x_{\ell})$, it turns out that

$$\phi^{-1}(\vec{x}) = \sum_{i=1}^{\ell} c_i x_i \pmod{N}$$

where, letting Q_i denote $P_i^{\alpha_i}$ and letting Q_{-i} denote $\prod_{j \neq i} P_j^{\alpha_j}$, the chinese remainder coefficients c_i are as follows:

$$c_i = Q_{-i} \cdot (Q_{-i}^{-1} \pmod{Q_i}).$$

Exercise: Check that this is indeed the inverse of ϕ .

3 The Multiplicative Group $\mathbb{Z}_N^*$

The multiplicative group of numbers mod N , denoted $\mathbb{Z}_N^*$, consists of the set

$$S = \{1 \leq a < N : \gcd(a, N) = 1\}$$

with multiplication mod N being the group operation.

Theorem 3. $\mathbb{Z}_N^*$ is a group. Operations in this group including multiplication and computing inverses can be done in time polynomial in the bit length of the numbers, namely $\text{poly}(\log N)$.

Proof. Left as an exercise; see also Angluin [2]. □

Example 4: Multiplicative groups

$$\begin{aligned} \mathbb{Z}_2^* &= \{1\} \\ \mathbb{Z}_3^* &= \{1, 2\} \\ \mathbb{Z}_4^* &= \{1, 3\} \\ \mathbb{Z}_5^* &= \{1, 2, 3, 4\} \\ \mathbb{Z}_6^* &= \{1, 5\} \\ \mathbb{Z}_7^* &= \{1, 2, 3, 4, 5, 6\}. \end{aligned}$$

Some further facts about $\mathbb{Z}_N^*$

- The order of $\mathbb{Z}_N^*$, the number of positive integers smaller than N that are relatively prime to it, is called the Euler totient function of N and is denoted by $\phi(N)$.
- If $N = \prod_i p_i^{\alpha_i}$ is the prime factorization of N , then

$$\phi(N) = \prod_i p_i^{\alpha_i-1} (p_i - 1).$$

- For every $a \in \mathbb{Z}_N^*$,

$$a^{\phi(N)} = 1 \pmod{N}.$$

This is called Euler's theorem, a direct consequence of Lagrange's theorem and the fact that the order of $\mathbb{Z}_N^*$ is $\phi(N)$. In the special case where the modulus P is prime, for every $a \in \mathbb{Z}_P^*$,

$$a^{P-1} = 1 \pmod{P},$$

a fact that is referred to as Fermat's little theorem.

Example 5: Determining $\phi(N)$

If N is prime, then $\phi(N) = N - 1$ and if $N = PQ$ is a product of two distinct primes, then $\phi(N) = (P - 1)(Q - 1)$.

4 The Multiplicative Group $\mathbb{Z}_P^*$ for a Prime P

Let's first focus on the case of $\mathbb{Z}_P^*$ when P is prime. Then, $\mathbb{Z}_P^* = \{1, 2, 3, \dots, P - 1\}$ and its order is $\phi(P) = P - 1$. The following is a very important property of $\mathbb{Z}_P^*$ when P is prime.

Theorem 4. *If P is prime, then $\mathbb{Z}_P^*$ is a cyclic group.*

Proof. We refer the reader to [2] for a proof. Note that it is very tempting to try to prove this theorem by using Theorem 2 which says that every group with prime order is cyclic. But the order of $\mathbb{Z}_P^*$ is $P - 1$, which is not necessarily prime. This is a very common mistake! $\square$

Example 6: $\mathbb{Z}_7^*$

$\mathbb{Z}_7^* = \{1, 2, 3, 4, 5, 6\} = \{5^6, 5^4, 5^5, 5^2, 5^1, 5^3\} = \{5^i \pmod{7} : i > 0\}$. So, 5 is a generator of $\mathbb{Z}_7^*$.

Even if N is not prime, $\mathbb{Z}_N^*$ may end up being cyclic: in particular, it is known that this happens exactly when $N = 1, 2, 4, p^k$ or $2p^k$ where p is an odd prime number.

One could ask several followup questions such as:

- how many generators are there for $\mathbb{Z}_P^*$?
- how to tell (efficiently) if a given element g is a generator?
- how to sample a random generator for $\mathbb{Z}_P^*$?

We will answer them in the sequel, starting with the first question.

4.1 $\mathbb{Z}_P^*$ has lots of generators

The proof of the following theorem is contained in the proof of Theorem 4 and can be found in [2].

Theorem 5. *The number of generators in $\mathbb{Z}_P^*$ is $\phi(P - 1)$.*

Do not confuse this with Theorem 2: although P is prime, the group $\mathbb{Z}_P^*$ has order $P - 1$, not P . Hence its number of generators is $\phi(P - 1)$, not $P - 1$.

Now, how large is $\phi(P - 1)$ asymptotically? This is answered by the following classical theorem.

Theorem 6. *For all sufficiently large integers N , $\phi(N) = \Omega(N / \log \log N)$.*

In other words, if you pick a random element of $\mathbb{Z}_P^*$, you will see a generator with probability

$$\frac{\phi(P - 1)}{P - 1} = \frac{\Omega((P - 1) / \log \log (P - 1))}{P - 1} = \Omega(1 / \log \log P),$$

so, reasonably often!

4.2 The multiplicative group $\mathbb{Z}_P^*$ and the additive group $\mathbb{Z}_{P-1}$

Let us note the following structural fact about $\mathbb{Z}_P^*$ before proceeding further. Fix a generator g of $\mathbb{Z}_P^*$. These two groups are isomorphic with an isomorphism ϕ that maps $x \in \mathbb{Z}_{P-1}$ to $g^x \in \mathbb{Z}_P^*$. In particular, consider

$$\phi(x) = g^x \pmod{P},$$

and note that it satisfies $\phi(x + y) = \phi(x) \cdot \phi(y)$.

The isomorphism is efficiently computable in the forward direction (exponentiation, using the repeated squaring algorithm) but not known to be efficiently computable in the reverse direction. The latter is the discrete logarithm problem (see Section 4.4).

Here is a quick application of this isomorphism:

Lemma 1. *Let P be an odd prime. If g is a generator of $\mathbb{Z}_P^*$, then so is g^x as long as x and $P - 1$ are relatively prime.*

Proof. Left as an exercise. □

As a corollary, we immediately derive the fact that $\phi(P - 1)$ elements of $\mathbb{Z}_P^*$ are generators (Theorem 5).

4.3 Primes, Primality Testing and Generating Random Primes

The prime number theorem tells us that there are “sufficiently many” prime numbers. In particular, letting $\pi(N)$ denote the number of prime numbers less than N , we know that

$$\pi(N) = \Omega(N / \log N).$$

Thus, if you pick a random number smaller than N , with probability about $1 / \log N$ (which is $1 / \text{polynomial}$ in the bit-length of the numbers in question) you have a prime number at hand.

The next question is how to recognize that a given number is prime. This has been the subject of extensive research in computational number theory with many polynomial-time algorithms, culminating with the deterministic polynomial-time primality testing algorithm of Agrawal, Kayal and Saxena [1] (a.k.a. AKS) in 2002.

An algorithm for finding a random prime. The above two facts put together tell us how to generate a random n -bit prime number—just pick a random number in $[2^{n-1}, 2^n)$ and test if it is prime (you can additionally sample only odd numbers). In expected n iterations of this procedure, you will find a n -bit prime number, even a random one at that.

How to tell if a given g is a generator of $\mathbb{Z}_P^*$? We know that $g^{P-1} = 1 \pmod{P}$ and we want to check if there is some smaller power of g that equals 1. We also know (by Lagrange) that any such power has to be a divisor of $P-1$. However, there can be super-polynomially many divisors of $P-1$ (as a function of $\log P$). It turns out, however, that you do not need to check all divisors, but rather only the terminal divisors. More precisely, let $P-1 = \prod_i q_i^{\alpha_i}$ be the prime factorization of $P-1$. Then, the following algorithm works, on input g and the prime factorization of $P-1$:

- For each i , check if $g^{(P-1)/q_i} \stackrel{?}{=} 1 \pmod{P}$. If yes for any i , say “not a generator” and otherwise say “generator”.

That’s nice. But can one tell if g is a generator given only g and a prime P (as opposed to the prime factorization of $P-1$ which is in general hard to compute)? We don’t know, so we have to find a way around it. There are two solutions:

Solution 1. Pick a prime $P = 2Q + 1$ where Q is prime. Such primes are called safe primes, and Q is called a Sophie-Germain prime after the famous mathematician. While there are infinitely many primes, it has only been conjectured that there are infinitely many Sophie-Germain primes. This remains unproven.

Solution 2. Pick a random prime P together with the prime factorization of $P-1$. This, it turns out, can be done due to an algorithm of Kalai [4].

Solution 1 is what people typically use in practice.

4.4 The Discrete Log Problem

We will present an algorithm for discrete logarithms that works over any group. This is the so-called baby-step giant-step algorithm that runs in time $O(\sqrt{|\mathbb{G}|} \log |\mathbb{G}|)$. It is still exponential but a square-root factor faster than naïve exhaustive search.

4.4.1 Solving the Discrete Log Problem

The problem. Given elements $g, h \in \mathbb{G}$, the problem is to find an x such that $h = g^x$ in $\mathbb{G}$.

The main idea. Since we know that $0 \leq x < |\mathbb{G}|$, let us write x as

$$x = x_1 y + x_0$$

where $y = \lceil \sqrt{|\mathbb{G}|} \rceil$, and $0 \leq x_0, x_1 < y$. In other words, write x in base- y . Now, we know that

$$h = g^{x_1 y + x_0} = (g^y)^{x_1} \cdot g^{x_0},$$

or, equivalently

$$h \cdot g^{-x_0} = (g^y)^{x_1}.$$

We now enumerate the possible values of x_0 and x_1 separately (recall that y is known to us since we've picked it). Doesn't this require $O(|\mathbb{G}|)$ work still? The answer is no. To see why, recall that $0 \leq x_0, x_1 < y$. Therefore, we now only need to enumerate $2y = O(\sqrt{|\mathbb{G}|})$ possible candidate values: all the values of hg^{-x_0} and $(g^y)^{x_1}$ respectively. We enumerate all of them, sort these two lists and then check (in time $O(y + y \log y)$) if there exists a tuple (x_0, x_1) for which they are equal. Given such a match, return $x = x_1 y + x_0$.

Can we do better? Baby-step giant-step gives a generic square-root-time algorithm for computing discrete logarithms. However, for $\mathbb{Z}_P^*$, better algorithms are known: the index-calculus algorithm runs in $2^{O(\sqrt{\log P \log \log P})}$ time, and the number field sieve algorithm that runs in $2^{O((\log P)^{1/3}(\log \log P)^{2/3})}$ time. Both are sub-exponential, but far from polynomial-time algorithms.

4.4.2 Random Self-Reducibility of Discrete Logarithms

How hard is the discrete logarithm problem for different choices of P, g and x ? Could it be that the problem is hard for a worst-case choice of these values, yet easy for many values, or random values? What we would ideally like is a worst-case to average-case reduction that tells us that if the discrete logarithm problem can be solved for (appropriately defined) random P, g and x , then it can be solved for every P, g and x . We unfortunately do not know such a statement, but we can prove something.

As a warmup, fix P and a generator g of $\mathbb{Z}_P^*$, and assume that you have an algorithm A that solves discrete log for $(P, g, g^x \pmod{P})$ for a random x . How do you construct out of A an algorithm W that solves discrete log for every $(P, g, g^x \pmod{P})$? The idea is to turn this instance of the discrete log problem into a random instance in such a way that a solution to the random instance can be mapped back into a solution for the given instance. The reduction is shown in Figure 1. The more general theorem (in which g is random as well, and not only x) is given below.

Theorem 7 (Random Self-Reducibility). *Fix a prime P . If discrete log over $\mathbb{Z}_P^*$ can be solved in polynomial-time for a uniformly random generator g and $x \in \mathbb{Z}_{P-1}$, then it can be solved for every generator g and $x \in \mathbb{Z}_{P-1}$.*

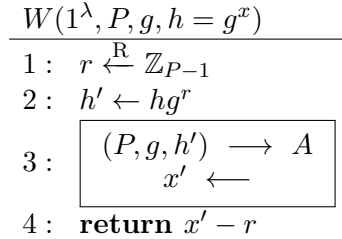


Figure 1: The toy version of random self-reduction for discrete log that randomizes only h .

Proof. Assume A (for average-case) is a $\text{poly}(n)$ -time algorithm that solves discrete log over $\mathbb{Z}_P^*$ for a $1/\text{poly}(n)$ fraction of (g, x) where $n = \log P$. We wish to demonstrate a $\text{poly}(n)$ -time algorithm W (for worst-case) that solves discrete log over $\mathbb{Z}_P^*$ for every (g, x) . The algorithm W , on input $(P, g, h = g^x \pmod{P})$, works as follows:

1. Pick a random s s.t. $\gcd(s, P-1) = 1$. Let $g' = g^s \pmod{P}$.
2. Pick a random $r \leftarrow \mathbb{Z}_{P-1}$ and compute $h' = h^s \cdot g^r \pmod{P}$.
3. Run $A(P, g', h')$ and let x' be its output.
4. Return $x = x' - s^{-1}r \pmod{P-1}$ as the discrete log solution to (P, g, h) .

Analysis. First, note that by our choice of s , g' is a generator as well. Indeed, it is a uniformly random generator of $\mathbb{Z}_P^*$. Now

$$h' = h^s \cdot g^r = g^{sx+r} \pmod{P}$$

which, by the random choice of r , is a uniformly random element of $\mathbb{Z}_P^*$ (even conditioned on g'). Thus, A will produce the discrete log x' of h' w.r.t. g' with probability $1/\text{poly}(n)$. In this event, we can compute x as

$$x = x' - s^{-1}r \pmod{P-1}.$$

If A refuses to solve the discrete log problem or it outputs an incorrect answer (either of which can happen with probability at most $1 - 1/\text{poly}(n)$), then W repeats this process again with fresh choice of s and r . Thus the reduction succeeds with probability $1 - \text{negl}(n)$ when run on $n \cdot \text{poly}(n)$ trials. $\square$

4.5 Quadratic Residues

Let us take a detour and look at a different facet of the equation

$$h = g^x \pmod{P},$$

namely the problem of computing g given h and x . In fact, we will look at the special case $x = 2$ for now. So, can you compute square roots mod P ? That is, given

$$h = g^2 \pmod{P},$$

can we efficiently recover a square root g of h ?

Before getting there, one could ask which numbers mod P are squares, namely which $h \in \mathbb{Z}_P^*$ can be written as $h = g^2 \pmod{P}$ for some $g \in \mathbb{Z}_P^*$. Such numbers are also called quadratic residues. Can you efficiently determine, given h , whether it is a quadratic residue?

Theorem 8. *Let P be an odd prime number and let g be any generator of $\mathbb{Z}_P^*$. The following conditions are equivalent.*

1. $\text{dlog}_g(h)$ is an even number;
2. h is a quadratic residue mod P ; and
3. $h^{(P-1)/2} = 1 \pmod{P}$.

Proof. We will show the following implications.

(1) $\Rightarrow$ (2). Assume $h = g^x \pmod{P}$ where x is an even number. Then, $h = (g^{x/2})^2$, so it is a quadratic residue.

(2) $\Rightarrow$ (3). Assume $h = t^2 \pmod{P}$ is a quadratic residue. Then,

$$h^{(P-1)/2} = t^{P-1} = 1 \pmod{P}.$$

where the second equation is by Fermat's little theorem (or Lagrange's theorem).

(3) $\Rightarrow$ (1). Assume $h^{(P-1)/2} = 1 \pmod{P}$. Letting $h = g^x$, we have

$$g^{x(P-1)/2} = 1 \pmod{P}.$$

This means that $P-1$ divides $x(P-1)/2$ which can only happen when x is even. $\square$

Since exactly half of the elements of $\mathbb{Z}_{P-1}$ are even (and these are all of the possible values of $\text{dlog}_g(h)$), it follows (from item 1 in the theorem) that exactly half the elements of $\mathbb{Z}_P^*$ are quadratic residues. If t is a square root of h , so is $-t$. Thus, every quadratic residue has two distinct square roots.

One can compute the two square roots of any quadratic residue in $\text{poly}(n) = \text{poly}(\log P)$ time. We will prove this next for a special class of P 's.

Theorem 9. Assume $P = 3 \pmod{4}$ and that h is a quadratic residue modulo P . The two square roots of $h \pmod{P}$ are $h^{(P+1)/4}$ and $-h^{(P+1)/4}$.

Proof. First of all, $(P+1)/4$ is an integer precisely because $P = 3 \pmod{4}$. Now, assume that $h = t^2 \pmod{P}$. Then,

$$h^{(P+1)/4} = t^{(P+1)/2} = t \cdot t^{(P-1)/2} = \pm t \pmod{P}.$$

The last equality is because $t^{P-1} = 1 \pmod{P}$ and $t^{(P-1)/2}$, being its square root, is either 1 or $-1 \pmod{P}$. $\square$

For the case of general P , an algorithm due to Berlekamp [3] computes square roots in probabilistic polynomial time.

4.6 Other Roots

If e is relatively prime to $P-1$, it is much easier to solve the equation $h = g^e \pmod{P}$ given h and e . It is not hard to verify that if $d = e^{-1} \pmod{P-1}$ (this exists because e is relatively prime to $P-1$), then

$$g = h^d \pmod{P},$$

is the unique solution. This is the basis of the RSA [5] cryptosystem: in RSA the modulus is composite, and the factorization provides the trapdoor needed to compute the corresponding inverse exponent.

4.7 The Diffie-Hellman Assumptions

Given g^x and $g^y \pmod{P}$, you can compute $g^{x+y} = g^x \cdot g^y \pmod{P}$, but can you compute $g^{xy} \pmod{P}$? If you can compute discrete logarithms, then you can compute x from g^x , and raise g^y to x to get $(g^y)^x = g^{xy} \pmod{P}$. But discrete log is hard, so this isn't an efficient way to solve the problem. Indeed, this problem, called the computational Diffie-Hellman (CDH) problem, appears to be computationally hard, though it is not known to be as hard as computing discrete logarithms.

Assumption 1 (Computational Diffie-Hellman Assumption). *For a random n -bit prime P and random generator g of $\mathbb{Z}_P^*$, and random $x, y \in \mathbb{Z}_{P-1}$, there is no polynomial (in n) time algorithm that computes $g^{xy} \pmod{P}$ given $P, g, g^x \pmod{P}, g^y \pmod{P}$.*

Moreover, it appears hard to even tell if you are given the right answer or not! But this requires some care to formalize. At first, one may think that given P, g, g^x, g^y , it is hard to distinguish between the right answer $g^{xy} \pmod{P}$ versus a random number $u \pmod{P}$. Let us call the assumption that this decisional problem is hard the decisional Diffie-Hellman (DDH) assumption.

Assumption 2 (First take: Decisional Diffie-Hellman Assumption). *For a random n -bit prime P and random generator g of $\mathbb{Z}_P^*$, and random $x, y \in \mathbb{Z}_{P-1}$ and a random number $u \in \mathbb{Z}_P^*$, there is no polynomial (in n) time algorithm that distinguishes between $(P, g, g^x \pmod{P}, g^y \pmod{P}, g^{xy} \pmod{P})$ and $(P, g, g^x \pmod{P}, g^y \pmod{P}, u \pmod{P})$.*

However, Assumption 2 turns out to be false as we will now show.

4.7.1 DDH is False in the multiplicative group

Let's step back and ask if the DDH assumption is actually true. It seems awfully strong on first look. It says not only that it is hard to compute g^{xy} from g^x and g^y , but also that the correct g^{xy} cannot be efficiently distinguished from a random group element.

We will now show that some information about g^{xy} indeed does leak from g^x and g^y . For this end, we will use the notion of quadratic residues in $\mathbb{Z}_P^*$. Recall that:

- exactly half the elements in $\mathbb{Z}_P^*$ are quadratic residues,
- these are precisely the elements h whose discrete logarithm with respect to any generator g is even, and
- we can decide in polynomial-time whether h is a quadratic residue or not.

Notice what this says: even though the discrete log x itself is hard to compute in its entirety, its parity can in fact be efficiently computed!

Armed with this observation, we prove the following theorem.

Theorem 10. *g^{xy} is a quadratic residue if and only if either g^x or g^y (or both) is a quadratic residue.*

Proof. g^x (resp. g^y) is a quadratic residue if and only if x (resp. y) is even (and therefore even mod $P-1$ since $P-1$ is itself necessarily even). Thus, if g^x or g^y is a quadratic residue, then $xy \pmod{P-1}$ is even and therefore g^{xy} is a quadratic residue as well. Conversely, if g^{xy} is a quadratic residue, then xy is even, so either x or y is even. In other words, either g^x or g^y is a quadratic residue. $\square$

Thus, given g^x and g^y , one can tell whether either of them is a quadratic residue and therefore whether g^{xy} should be a quadratic residue. This immediately translates to an algorithm that distinguishes between g^{xy} and a uniformly random element mod P .

Thus, we need to refine our assumption. Looking at the core reason behind the above attack, we see that $\mathbb{Z}_P^*$ contains a proper subgroup that is efficiently recognizable: the subgroup QR_P of quadratic

residues. Given g^x , we can efficiently test whether $g^x \in \text{QR}_P$, which reveals the parity of x . Hence, given g^x and g^y , we can determine whether xy is even, and therefore whether $g^{xy} \in \text{QR}_P$. This is what enables the attack.

A natural solution is to work in a group with no non-trivial proper subgroups, namely of prime order. One example of such a group is the subgroup of quadratic residues in $\mathbb{Z}_P^*$, denoted QR_P , where $P = 2Q + 1$ is a safe prime. This subgroup QR_P has order

$$\frac{P-1}{2} = Q,$$

which is prime. Therefore, every non-identity element of QR_P is a generator, and QR_P has no non-trivial proper subgroups. In particular, the previous quadratic-residuosity test is now useless, since every element we work with already lies in QR_P .

With this change, we can state the following DDH assumption, which is widely believed to be true.

Assumption 3 (Decisional Diffie-Hellman Assumption). *Let $P = 2Q + 1$ be a random n -bit safe prime and let QR_P denote the subgroup of quadratic residues in $\mathbb{Z}_P^*$. For a random generator g of QR_P , and random $x, y \in \mathbb{Z}_Q$ and a random number $u \in \text{QR}_P$, there is no polynomial (in n) time algorithm that distinguishes between $(P, g, g^x \pmod{P}, g^y \pmod{P}, g^{xy} \pmod{P})$ and $(P, g, g^x \pmod{P}, g^y \pmod{P}, u \pmod{P})$.*

We know that $\text{DLOG} \Rightarrow \text{CDH} \Rightarrow \text{DDH}$ but no implications are known in the reverse directions.

References

- [1] M. Agrawal, N. Kayal, and N. Saxena. Primes is in p. *Annals of mathematics*, pages 781–793, 2004.
- [2] D. Angluin. Lecture notes on the complexity of some problems in number theory. 1982.
- [3] E. R. Berlekamp. Factoring polynomials over large finite fields. *Mathematics of computation*, 24(111):713–735, 1970.
- [4] A. Kalai. Generating random factored numbers, easily. *Journal of Cryptology*, 16(4):287–289, 2003.
- [5] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.